



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 6: Thiết kế chi tiết

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Chương này tập trung trình bày một số chủ đề sau đây:

- 6.1. Giới thiệu
- 6.2. Xây dựng mô hình lớp thiết kế
- 6.3. Xây dựng lược đồ CSDL

I 6.1 GIỚI THIỆU

- Việc thiết kế chi tiết liên quan đến việc chuyển đổi mô hình phân tích theo khái niệm thành các lớp cài đặt được theo chiến lược đặt ra trong mô hình thiết kế hệ thống.
- Thiết kế chi tiết thường bao gồm các hoạt động sau đây:
 - **Xây dựng mô hình lớp thiết kế:** Bước này chúng ta tập trung vào thiết kế các lớp thuộc cụm nghiệp vụ bao gồm các lớp thực thể trong miền bài toán và các lớp hỗ trợ cần thiết khác. Các lớp và trường của cụm nghiệp vụ có thể có được bằng cách sử dụng mô hình lớp phân tích.
 - **Xây dựng lược đồ cơ sở dữ liệu:** Quyết định cách lưu trữ dữ liệu và thiết kế khuôn dạng lưu trữ. Dữ liệu lưu trữ là dữ liệu không bị mất đi khi hệ thống dừng hoạt động.

| Mục đích thiết kế chi tiết

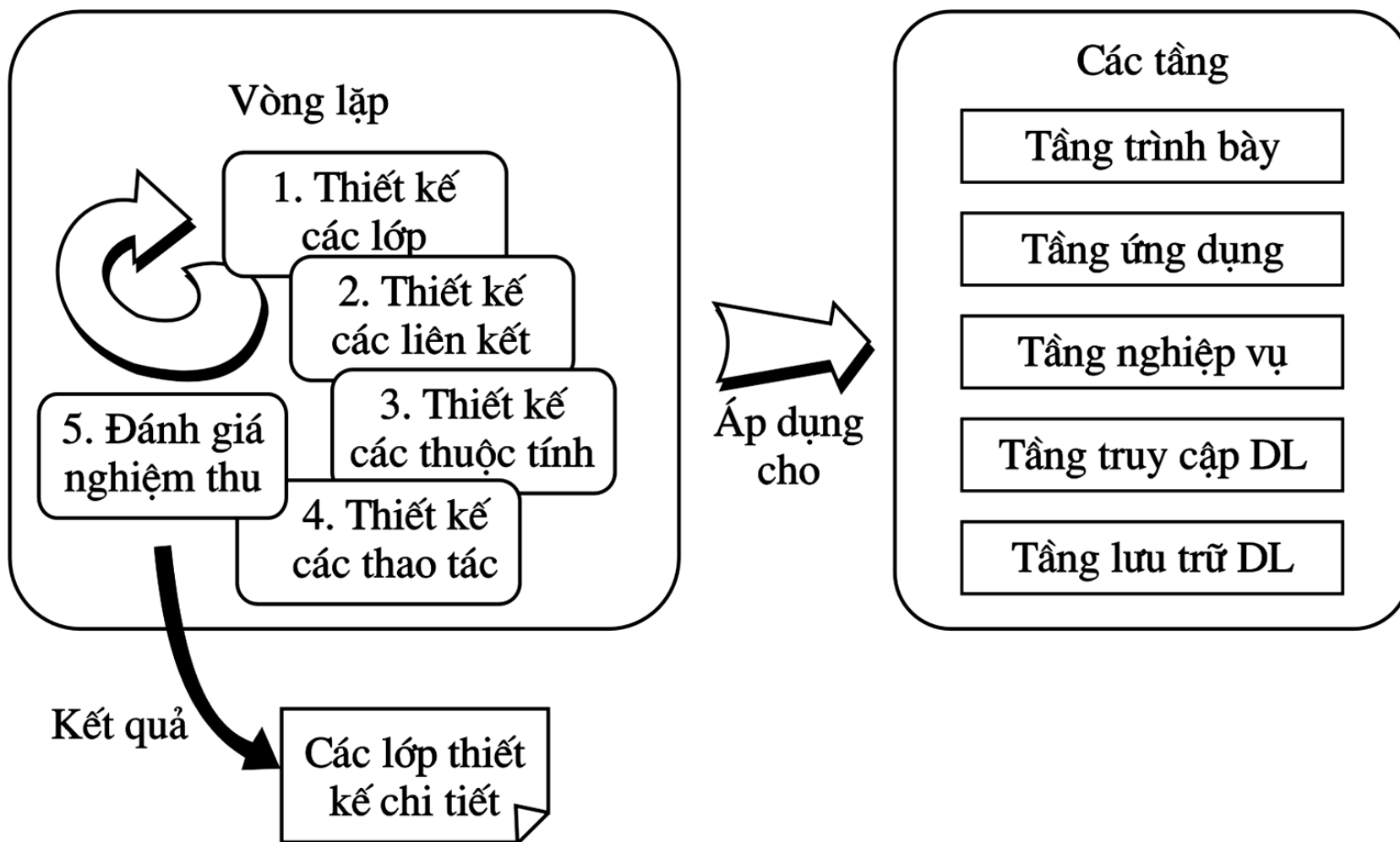
- Trong phân tích, ta tập trung nghiên cứu cấu trúc logic của các thông tin, cần thiết cho việc xây dựng một giải pháp nghiệp vụ.
 - Phân tích luôn được triển khai theo **nhãn quan ứng dụng**, và chưa tính tới các điều kiện về công nghệ.
- Trong thiết kế, đặc biệt là thiết kế chi tiết, ta cần nghiên cứu các cách tốt nhất để cài đặt cấu trúc logic nói trên, nhằm tối ưu hoá hiệu năng của ứng dụng.
 - Thiết kế phải được triển khai theo nhãn quan kỹ thuật và **phụ thuộc vào các khả năng và các hạn chế của tài nguyên tính toán và công nghệ lập trình phát triển ứng dụng**.

| Mục đích thiết kế chi tiết

- Đầu vào của việc thiết kế chi tiết là **các mô hình đã được thiết lập** từ các bước phân tích và thiết kế hệ thống trước đây, bao gồm:
 - Các mô hình về cấu trúc (các biểu đồ lớp, thành phần, hệ thống con, kiến trúc phân tầng).
 - Các mô hình hành vi (các biểu đồ ca sử dụng, tương tác, trạng thái và giao diện người dùng)
- Kết quả đầu ra của thiết kế chi tiết phải là một **mô hình sẵn sàng cho lập trình** (được diễn tả thông qua sơ đồ UML), thích hợp với ngôn ngữ lập trình và môi trường cài đặt đã được lựa chọn (hay sẵn có).

I Quy trình thiết kế chi tiết

Thiết kế chi tiết được tiến hành theo một quy trình lặp được diễn tả như hình sau:



I 6.2 XÂY DỰNG MÔ HÌNH LỚP THIẾT KẾ

- Khi chuyển từ pha phân tích sang pha thiết kế, có thể một số lớp sẽ bị loại bỏ (ví dụ như một số lớp điều khiển) và một số lớp khác sẽ được thêm vào (như các lớp để thực thi đa nhiệm hay các lớp từ các mẫu thiết kế).
- Người thiết kế sẽ tự quyết định thiết kế các đối tượng nghiệp vụ, biên và điều khiển sao cho có thể biến thành mã trình thực thi được.
- Đối với mỗi lớp thiết kế đưa ra, chúng ta cần chọn tên và kiểu của các trường. Thông thường, các trường được dẫn xuất từ các thuộc tính, các quan hệ liên kết cũng như từ quan hệ kế thừa được tìm thấy trong suốt quá trình phân tích.
- Quan hệ kế thừa không nhất thiết phải được ánh xạ thành cái gì mới mà cần quyết định xem là giữ hay không giữ chúng. Lý do là phân tích nhằm phát hiện ra các thực thể nghiệp vụ còn thiết kế nhằm hướng đến tính hiệu quả trong cài đặt sau này.

| 6.2.1 Ánh xạ các phương thức

- Khi chuyển đến giai đoạn thiết kế, chúng ta thường sử dụng các thuật ngữ phương thức (method) và thông điệp (message) trong lập trình hơn là thuật ngữ thao tác (operation) trong UML.
- Phương thức được đưa ra chỉ đơn thuần như một cách ghi lại hiện thực hóa các ca sử dụng và có thể dùng để kiểm định các lớp phân tích có hỗ trợ cài đặt hay không. Như vậy, một số phương thức gán cho các lớp trong pha phân tích có thể bị bỏ qua và một số phương thức mới lại được đưa vào.
- Trong nhiều trường hợp, các thông điệp sẽ được thêm bởi một số lý do sau đây:
 - Để cho phép các đối tượng client đọc hoặc thay đổi các giá trị của các trường.
 - Để cho phép đối tượng client truy nhập dữ liệu dẫn xuất. Ví dụ, như một thông điệp để đọc hóa đơn các mặt hàng đăng ký mua, chúng ta cũng muốn có thể đọc được tổng hóa đơn.
 - Nhờ kinh nghiệm hay trực giác nói rằng thông điệp thêm vào là có ích.
 - Bởi vì khuôn mẫu hay mẫu thiết kế mà chúng ta định sử dụng đòi hỏi phải có các thông điệp nào đó.

| 6.2.2 Phạm vi của các trường

- Chúng ta phải khai báo phạm vi truy nhập của trường. Phạm vi chỉ rõ các đoạn mã trình nào cho phép đọc hay thay đổi giá trị của các trường. Có bốn cách truy nhập sau đây:
 - **private** (ký hiệu dấu -) chỉ sử dụng trong phạm vi của lớp xác định.
 - **package** (ký hiệu dấu ~): dùng trong phạm vi lớp xác định và tất cả những lớp cùng gói.
 - **protected** (ký hiệu dấu #): dùng trong phạm vi lớp xác định, tất cả những lớp cùng gói và tất cả những lớp con của lớp định nghĩa (dù trong hay ngoài gói).
 - **public** (ký hiệu dấu +): dùng bất cứ ở đâu.

| Phạm vi của thông điệp

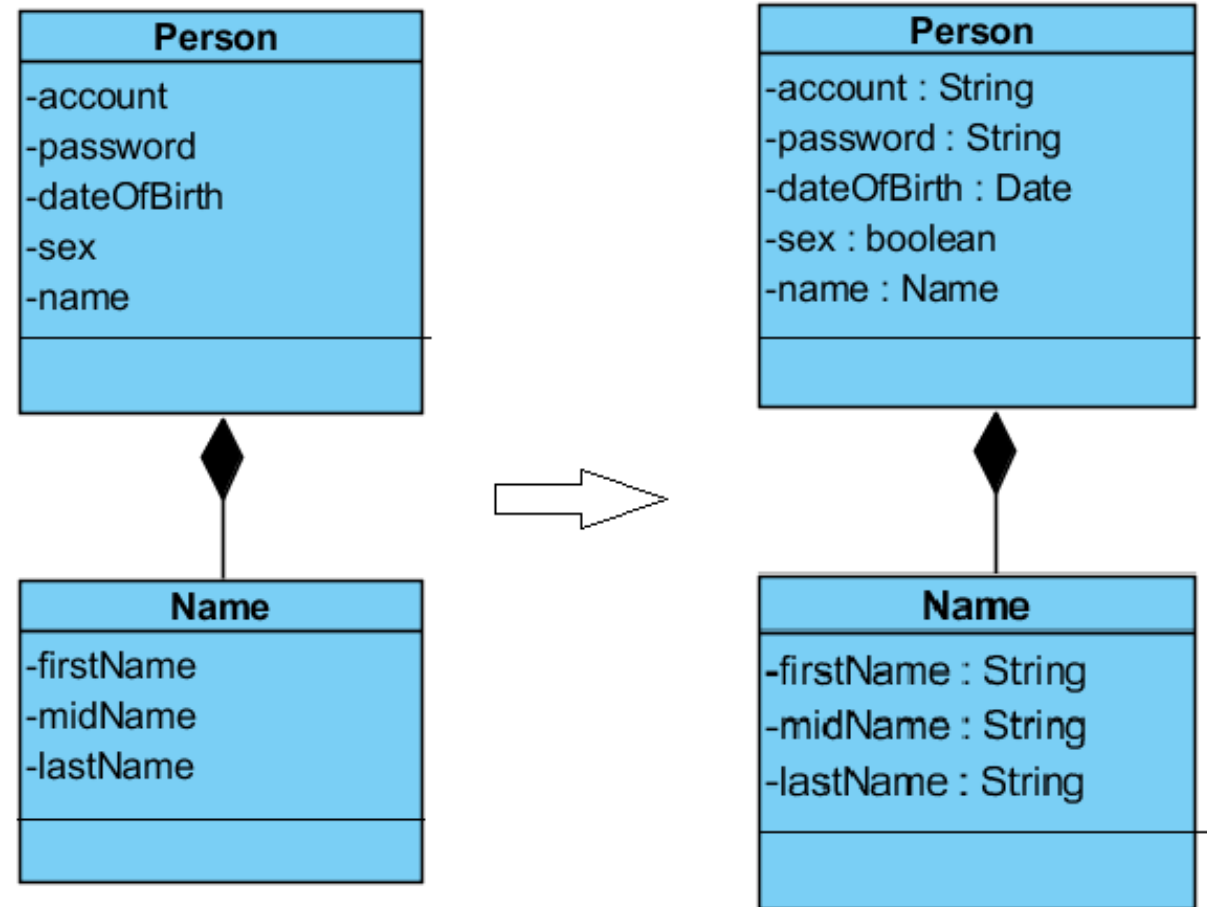
- Phạm vi cũng có thể áp dụng với thông điệp. Khi đó, thông điệp có thể là:
 - **public**: nếu nó là phần của giao diện của package.
 - **package**: nếu nó là mã cài đặt được sử dụng bởi chính lớp đó và bởi lớp trong cùng package.
 - **protected**: nếu nó là mã cài đặt được sử dụng bởi chính lớp đó, lớp con và các lớp trong cùng package.
 - **private**: nếu nó là mã cài đặt để sử dụng chỉ bởi chính lớp đó.

I 6.2.3 Ánh xạ các lớp, thuộc tính và kiểu quan hệ hợp thành

- Trong UML, một số ký hiệu giống nhau được sử dụng cả cho biểu đồ lớp phân tích và biểu đồ lớp thiết kế. Khi ánh xạ sang biểu đồ lớp thiết kế, ba vấn đề chính cần phải quan tâm:
 - Làm sao các lớp thiết kế phải thể hiện cài đặt được.
 - Thêm vào các thuộc tính và các kiểu thuộc tính.
 - Ánh xạ các quan hệ lớp để thể hiện được trong ngôn ngữ lập trình.
- Trong pha phân tích, quan hệ hợp thành là song hướng: bắt đầu từ một trong hai đối tượng, chúng ta có thể dễ dàng tìm thấy các đối tượng ở đầu kia. Tuy nhiên, khi chuyển sang thiết kế, chúng ta phải đối phó với một thực tế là các trường chỉ có thể hướng theo một chiều tại thời gian chạy. Nghĩa là, chúng ta có thể lấy trường từ một đối tượng sở hữu tới đối tượng ở đầu kia, nhưng không là ngược lại.
- Vì vậy, chúng ta cần quyết định xem chúng ta có muốn có một trường ở mỗi đầu của mỗi quan hệ hoặc chỉ ở một đầu và nếu vậy thì đầu nào.
- Cái được hợp thành là chủ sở hữu của mỗi quan hệ và như vậy cái hợp thành tương tự như một thuộc tính. Nghĩa là, đối tượng được hợp thành sử dụng dịch vụ của các thuộc tính hoặc các đối tượng hợp thành, nhưng không theo chiều ngược lại.

| 6.2.3 Ánh xạ các lớp, thuộc tính và kiểu quan hệ hợp thành

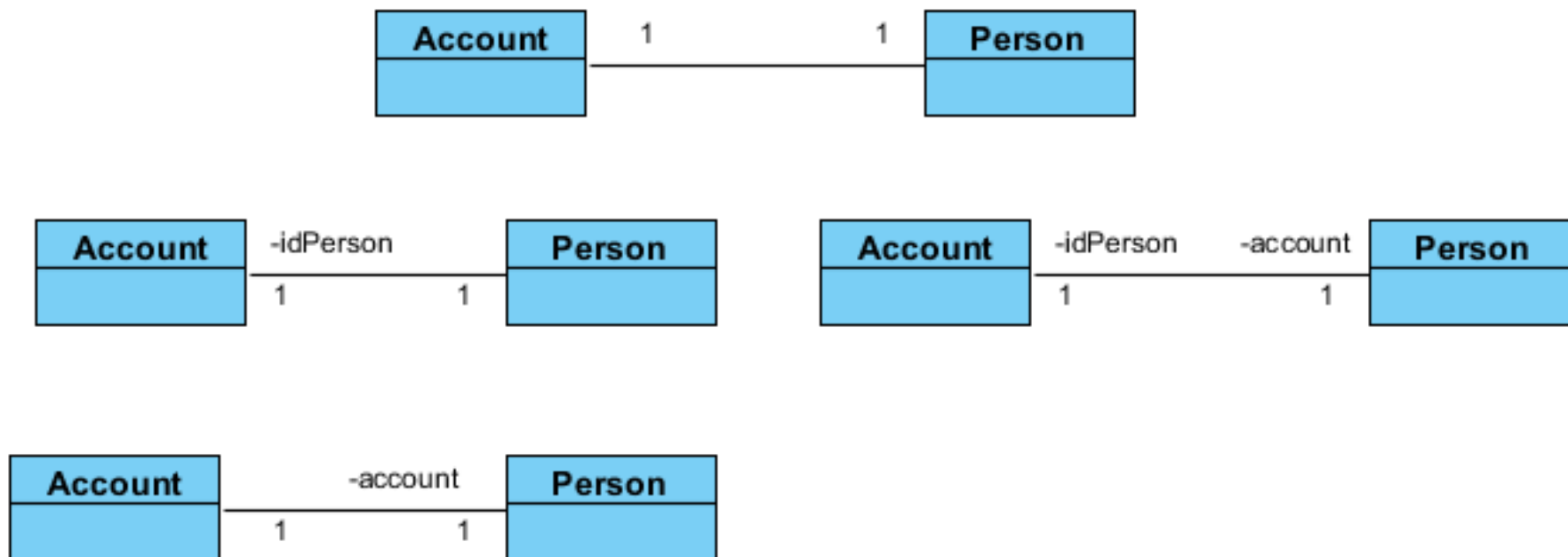
- Quan hệ hợp thành của lớp người **Person** và lớp họ tên **Name**, ta thêm thuộc tính **name:Name** bên trong **Person**.



| 6.2.4 Ánh xạ các kiểu quan hệ khác

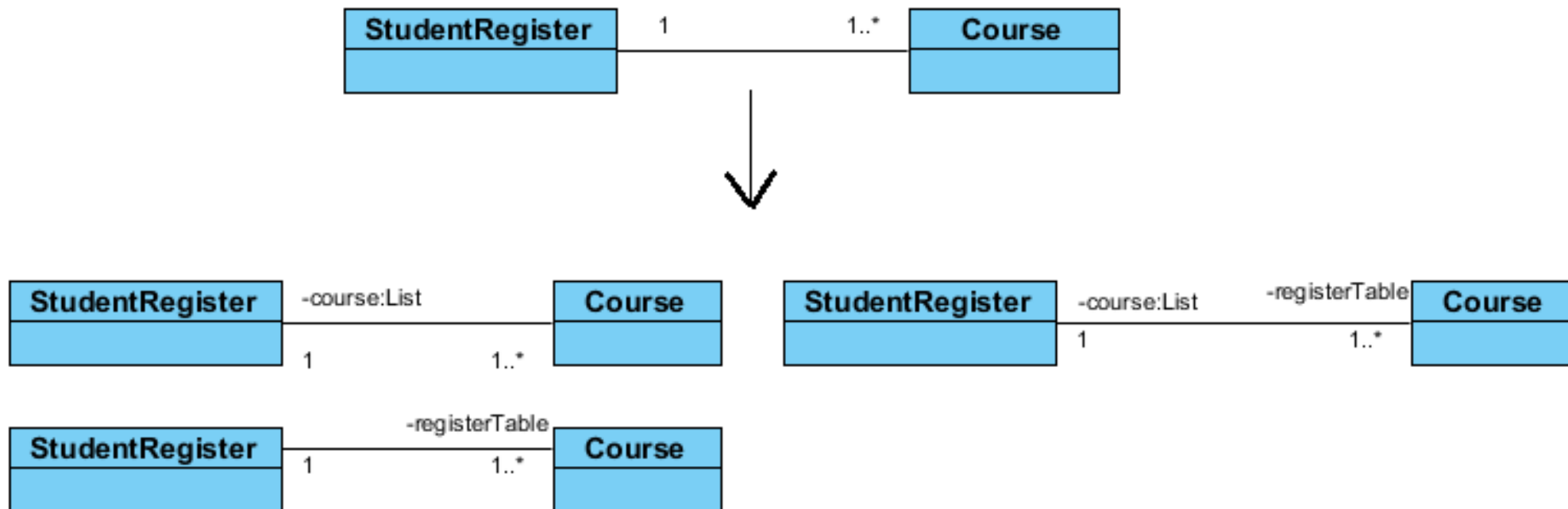
- Như đã trình bày trong Chương 4, có ba loại quan hệ giữa các lớp chính: *liên kết*, *kết hợp* và *hợp thành*. Do mục đích là ánh xạ sang thiết kế, chúng ta không cần phân biệt giữa liên kết và kết hợp vì các ngôn ngữ lập trình hướng đối tượng không có sự phân biệt này.
- Đa số liên kết mà chúng ta quyết định giữ lại từ phân tích, cùng với bất kỳ liên kết nào mà chúng ta thêm vào khi thiết kế thì chỉ như các trường trong các lớp.
- Vì các trường chỉ cho phép chuyển theo một hướng nên chúng ta cần phải quyết định xem chúng ta muốn đi theo cả hai hướng hay một hướng.
- Cách chúng ta cài đặt liên kết phụ thuộc vào tính nhiều ở mỗi điểm cuối: **1-1**, **1-n** hay **m-m**.

| Liên kết kiểu 1-1



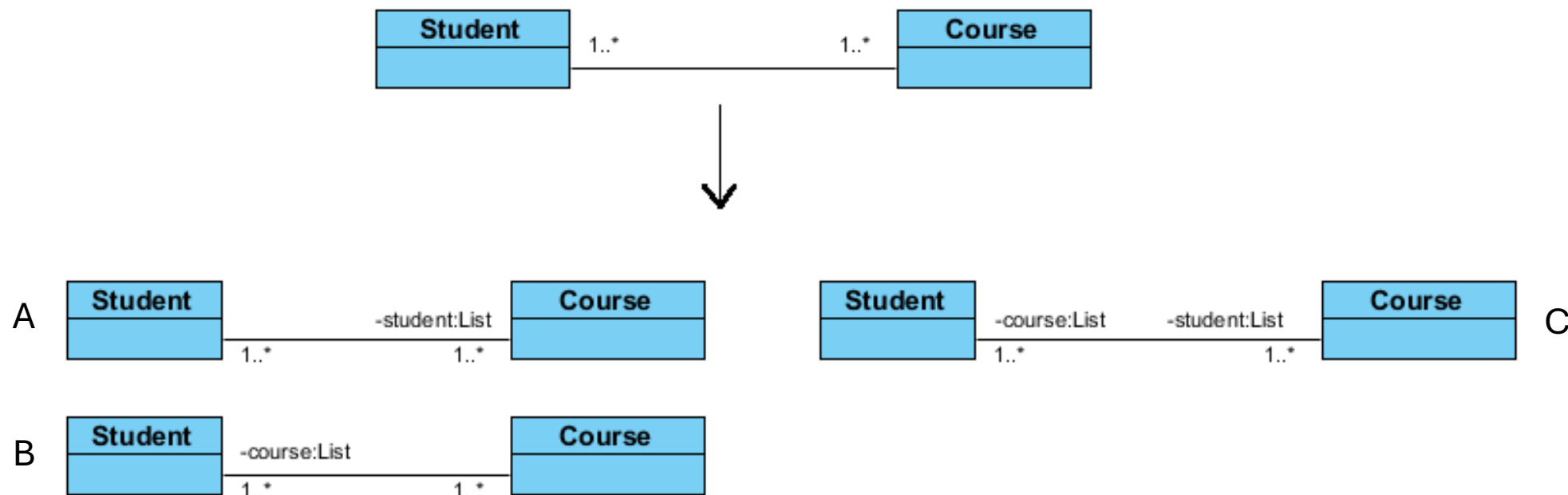
- Hãy xem phần biểu đồ lớp ở trên. Có ba cách cài đặt kiểu liên kết này:
 - Đặt trường *account* vào trong *Person*;
 - Đặt một trường gọi *idPerson* vào trong *Account*;
 - Hoặc kết hợp cả hai (two-way)
- Như vậy, ta có thể ánh xạ liên kết thành một chiều hay hai chiều nhưng phần lớn chúng ta nên chọn một chiều (one-way) vì cách 2 chiều (two-way) đôi phải có quá trình đồng bộ khi dữ liệu thay đổi.

| Liên kết kiểu 1-n



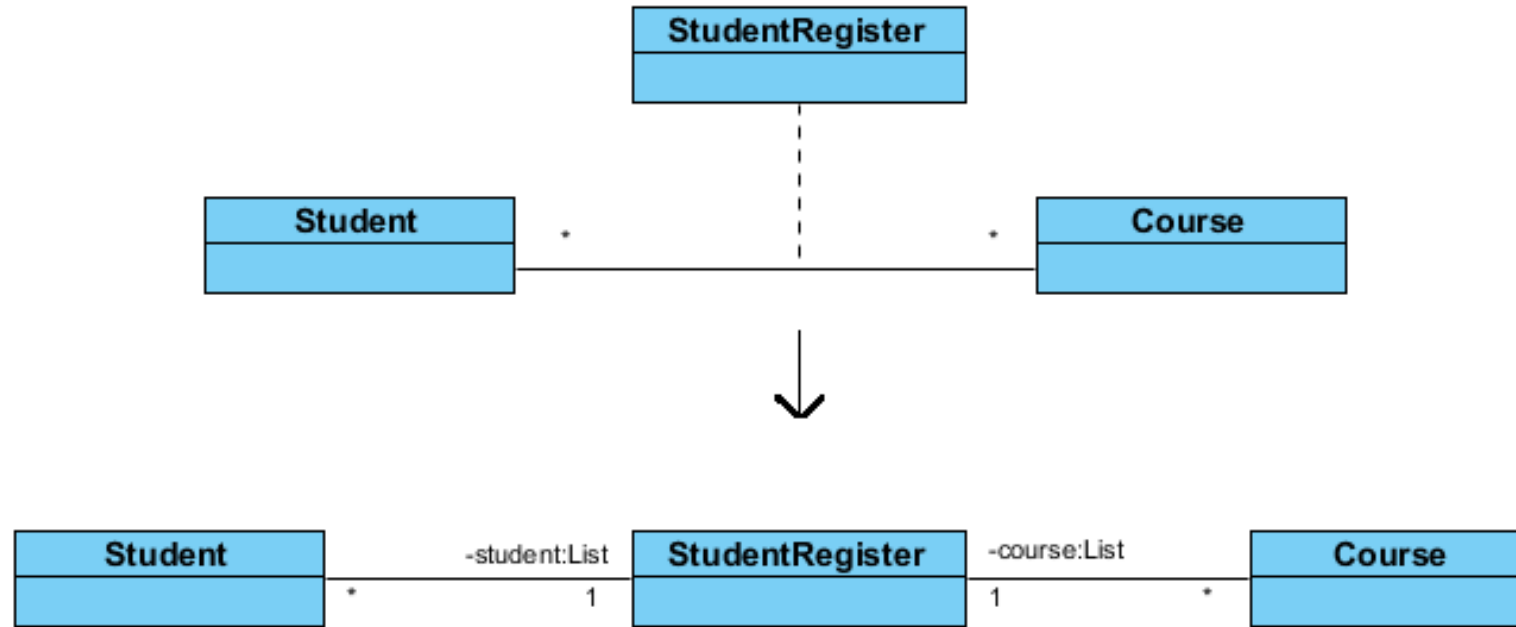
- Với trường hợp này, chúng ta phải quyết định có nên đặt một trường trong một đầu hoặc cả hai và những đối số tương tự cung cấp như trong trường hợp 1-1.
- Tuy nhiên, không giống như trường hợp 1-1, nếu chúng ta quyết định đặt một trường vào mỗi đầu cuối, chúng ta phải chuẩn bị để lưu trữ nhiều hơn một đối tượng liên quan.

| Liên kết kiểu n-m



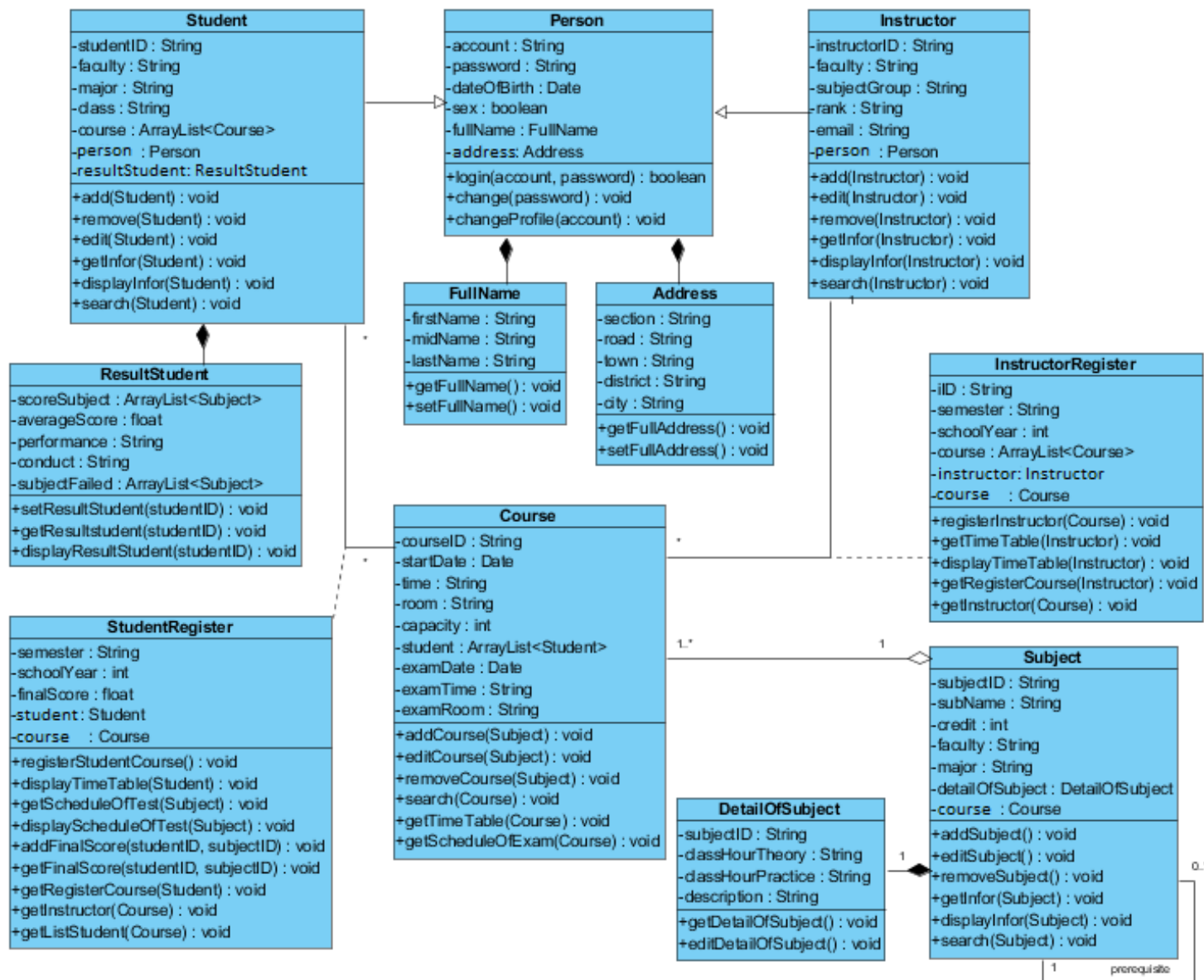
- Thường với liên kết nhiều-nhiều **n-m** thì thực sự không có chủ sở hữu của sự liên kết hợp trong trường hợp này
- Có một đối tượng **s:Student** có thể thuộc nhiều đối tượng **Course** và ngược lại. Xét ba trường hợp: A, B và C:
 - A: Tốt khi chúng ta quyết định bắt đầu với một **Student** và sau đó chuyển hướng tới các đối tượng **Course**.
 - B: Tốt nếu chúng ta quyết định bắt đầu với một **Course**.
 - C: Phù hợp nếu bắt đầu với một **Course** hoặc một **Student** -> vấn đề đồng bộ hóa.

| Các lớp liên kết



- Từ biểu đồ này, chúng ta biết các đối tượng sinh viên **Student** có thể đăng ký một số đối tượng môn học **Course** và các đối tượng môn học **Course** có thể được đăng ký bởi một số đối tượng **Student**.
- Nhưng với mỗi liên kết **StudentRegister**, ta cần ghi lại **ngày** và **trạng thái đăng ký**. Các dữ liệu này không thuộc về **Student** hoặc **Course**, mà thuộc về lớp liên kết. Do đó, chúng ta đưa ra **StudentRegister** như một lớp liên kết với các thuộc tính thích hợp.

Các lớp cụm nghiệp vụ cho Hệ quản lý học tín chỉ



| 6.3 XÂY DỰNG LƯỢC ĐỒ CƠ SỞ DỮ LIỆU

6.3.1 Các hệ quản trị cơ sở dữ liệu

- Cơ sở dữ liệu quan hệ là một loại cơ sở dữ liệu lưu trữ cung cấp quyền truy cập vào các điểm dữ liệu có liên quan đến nhau. Cơ sở dữ liệu quan hệ dựa trên mô hình dữ liệu quan hệ, một cách trực quan, đơn giản để biểu diễn dữ liệu trong bảng.
- Các thành phần của CSDL quan hệ:
 - **Table**(Bảng dữ liệu): Field, Row, Cell, Primary Key, Foreign Key...
 - **Relationship** (Mối quan hệ)
 - 1-1: Mỗi bảng ghi chỉ có một và một bảng tương ứng mà thôi.
 - 1-n: 1 bảng ghi ở bảng này có thể có nhiều bảng ghi tương ứng ở bảng kia.
 - n-n: 1 bảng ghi ở bảng này có thể có nhiều bảng ghi tương ứng ở bảng kia và ngược lại.
 - **Entity Relationship Diagram – ERD** (Lược đồ thể hiện mối quan hệ) là một cách giúp hiểu nhanh hơn về cấu trúc và cơ sở dữ liệu, dễ dàng thao tác hơn.
 - **Index** (Chỉ mục)
 - **SQL** (Structured Query Language) ...

| Database Management System: Hệ quản trị Cơ sở dữ liệu

- Database Management System (DBMS) là phần mềm giúp quản lý và vận hành cơ sở dữ liệu như: MySQL, SQL Server, Oracle Database....

DMBS	Kích thước hệ thống	Hệ điều hành	Năm ra đời
Oracle	Lớn	UNIX, Windows	1979
DB2	Lớn và vừa	Linux, Solaris	1985
MySQL	Vừa và nhỏ	All OS	1987
MS SQL Server	Vừa và nhỏ	Linux, Windows	2000
PostgreSQL	Vừa và nhỏ	UNIX, Windows	1986

I Cơ sở dữ liệu phi quan hệ

- Cơ sở dữ liệu phi quan hệ hay Non SQL hoặc còn được biết đến là NoSQL (Not Only SQL) là một loại CSDL được thiết kế để lưu trữ, truy vấn và quản lý dữ liệu không tuân theo cấu trúc bảng quan hệ truyền thống. NoSQL đáp ứng nhu cầu về tính linh hoạt, mở rộng và hiệu suất trong xử lý dữ liệu lớn.
- Các loại cơ sở dữ liệu phi quan hệ:
 - **Cơ sở dữ liệu tài liệu** (Document Databases): Lưu trữ dữ liệu dưới dạng tài liệu, thường là JSON, BSON hoặc XML. Mỗi tài liệu chứa một cấu trúc dữ liệu phức tạp và linh hoạt. Ví dụ: MongoDB, CouchDB.
 - **Cơ sở dữ liệu cặp giá trị - chìa khóa** (Key - Value Stores): Lưu trữ dữ liệu dưới dạng cặp chìa khóa và giá trị, đơn giản và nhanh chóng truy xuất. Ví dụ: Redis, DynamoDB.
 - **Cơ sở dữ liệu đồ thị** (Graph Databases): Chuyên lưu trữ và truy vấn dữ liệu có quan hệ phức tạp, như mạng xã hội hay các mối quan hệ phân cấp. Ví dụ: Neo4j, JanusGraph.
 - **Cơ sở dữ liệu cột** (Column-Family Stores): Tổ chức dữ liệu dưới dạng cột thay vì hàng, phù hợp với các ứng dụng cần xử lý khối lượng lớn dữ liệu. Ví dụ: Cassandra, HBase.

| 6.3.2 Ánh xạ các lớp thực thể

Sự tương đương giữa Lớp và Bảng

Mô hình đối tượng	Mô hình quan hệ
Lớp	Bảng
Thuộc tính có kiểu đơn	Cột
Thuộc tính có kiểu phức tạp	Các cột, hay Bảng và khóa ngoại
ID	Khoá chính
Liên kết và Kết nối	Khoá ngoại hay bảng
Thừa kế	Khoá chính đồng nhất trên nhiều bảng

| Ánh xạ các lớp thực thể

- Để ánh xạ một thực thể (đối tượng nghiệp vụ) từ mô hình hướng đối tượng thành lược đồ quan hệ, ta đưa ra bảng có cùng tên như lớp thực thể, trong đó mỗi dòng biểu diễn một đối tượng duy nhất của miền nghiệp vụ.
- Với trường đơn giản (kiểu cơ bản hay String), ta thêm một cột vào bảng với cùng tên như trường và kiểu dữ liệu thích hợp SQL.
- Để tiện cho lập trình hướng đối tượng, ta đưa ra một thuộc tính **ID** có thể kiểu nguyên để làm khóa chính. Lợi ích của việc thêm **ID** là nó làm đơn giản việc code và làm cho việc đọc các đối tượng hoặc chuyển từ đối tượng này sang đối tượng khác dễ dàng và hiệu quả hơn.

Address
-section : String
-road : String
-town : String
-district : String
-city : String



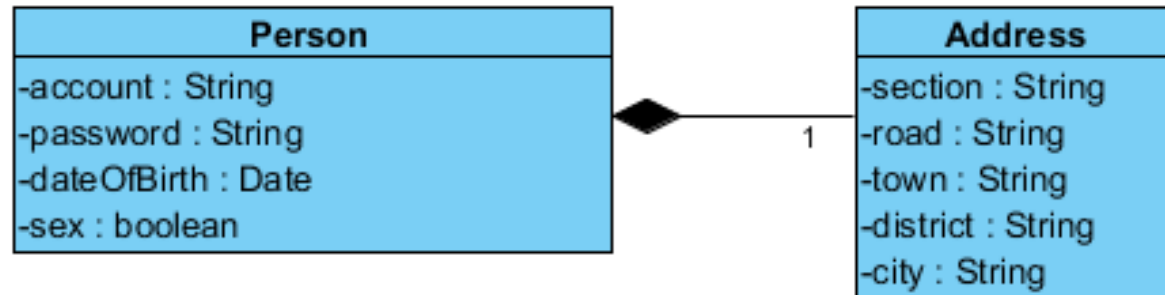
ADDRESS			
	Column Name	Data Type	Allow Nulls
?	ADDRESSID	int	☐
	SECTION	varchar(50)	☐
	ROAD	varchar(50)	☐
	TOWN	varchar(50)	☐
	DISTRICT	varchar(50)	☐
	CITY	varchar(50)	☐
			☐

| 6.3.3 Ánh xạ các liên kết

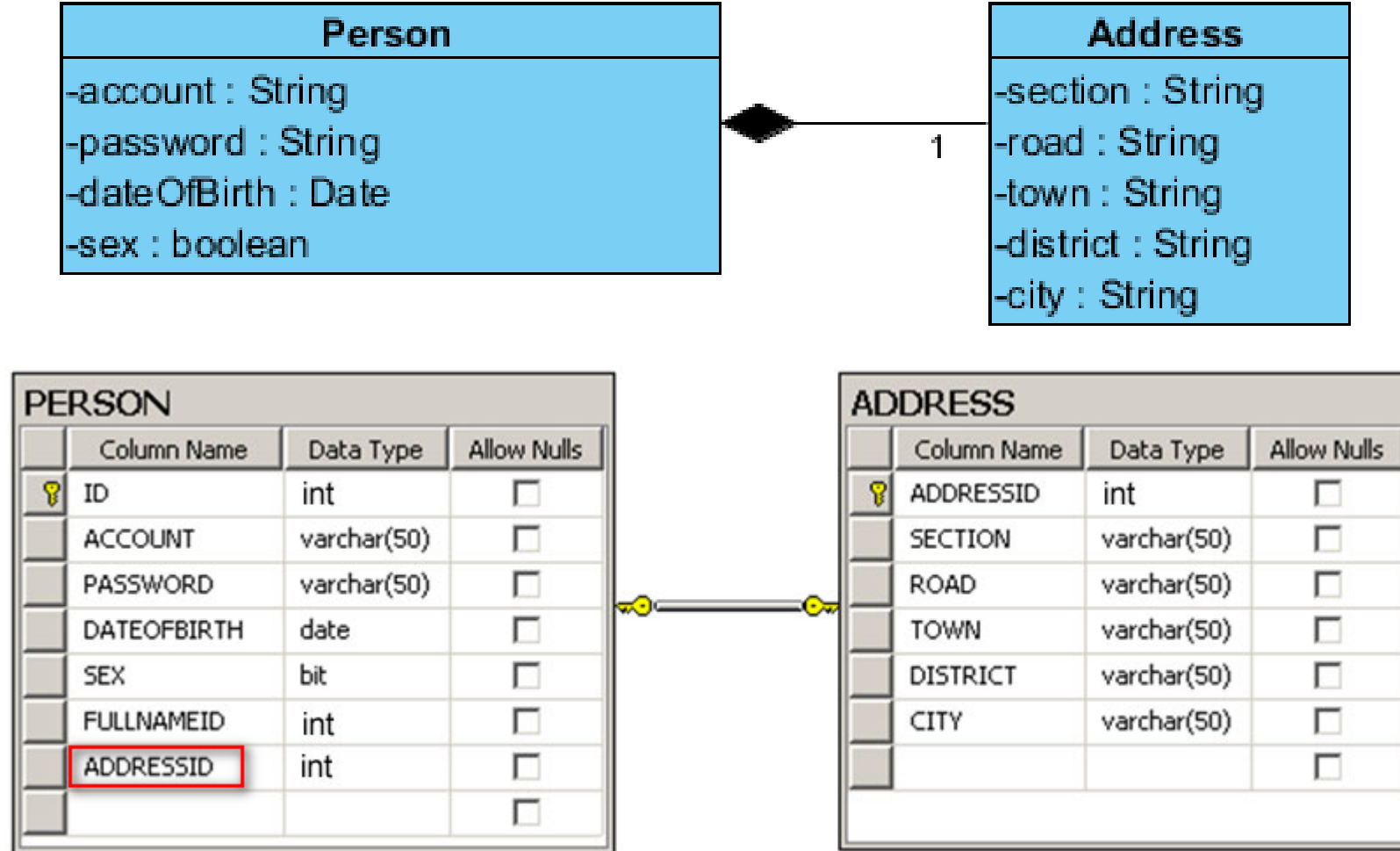
- Khi ánh xạ từ mô hình lớp phân tích thành mô hình lớp thiết kế, ta phải chuyển liên kết hai chiều trong phân tích thành liên kết một chiều. Tuy nhiên, vì cơ sở dữ liệu quan hệ lưu trữ trực tiếp liên kết hai chiều nên chúng ta không gặp phải vấn đề như khi mô hình lớp thiết kế.
- Ví dụ, nếu lược đồ CSDL cho phép liên kết từ thực thể A đến thực thể B, thì DBMS và ngôn ngữ truy vấn sẽ cho phép ta liên kết từ B đến A. Do đó, lược đồ cơ sở dữ liệu của ta giống với mô hình lớp phân tích hơn là mô hình lớp thiết kế.
- Các loại liên kết:
 - Liên kết 1-1
 - Liên kết 1-n
 - Liên kết n-m
 - Các lớp liên kết

| Liên kết 1-1

- Với liên kết 1 -1 chúng ta có thể **thêm một khóa ngoại vào một trong các bảng thực thể**. Một khóa ngoại là một thực thể trong một bảng tham chiếu đến khoá chính của một bảng khác. Nghĩa là, nó là một tham chiếu từ một dòng trong một bảng đến một dòng trong một bảng khác.
- Trong hình sau, Bảng **PERSON** chứa khóa ngoại **ADDRESSID** cho phép chúng ta tìm chi tiết của **PERSON** trong bảng **ADDRESS**. Chúng ta cũng có thể thêm khóa ngoại **PERSONID** vào bảng **ADDRESS** nhưng do tính chất hai chiều của cơ sở dữ liệu quan hệ, ta không cần đưa khóa ngoại vào hai bảng.
- Một hướng giải quyết khác là thay thế khóa ngoại bằng cách gộp hai bảng thành một bảng nhưng điều này chỉ nên thực hiện khi một bảng không được biểu diễn như một thực thể đúng nghĩa.

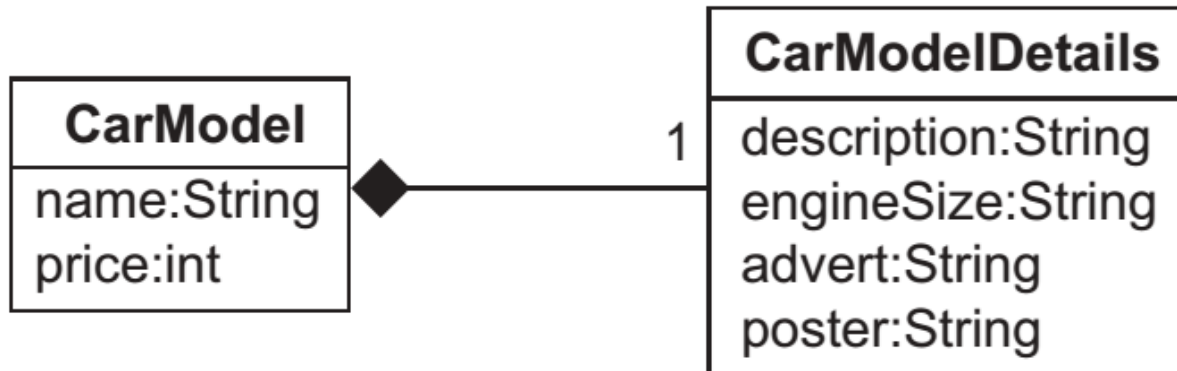


| Liên kết 1-1



Trong trường hợp có một liên kết tùy chọn như tính nhiều, 1 và 0..1, chúng ta có thể thêm một khoá ngoại vào tùy chọn. Cơ sở dữ liệu quan hệ cũng hỗ trợ cột có thuộc tính null nhưng để đơn giản tốt hơn nên tránh dùng.

| Liên kết 1-1 (Ví dụ)



CARMODEL

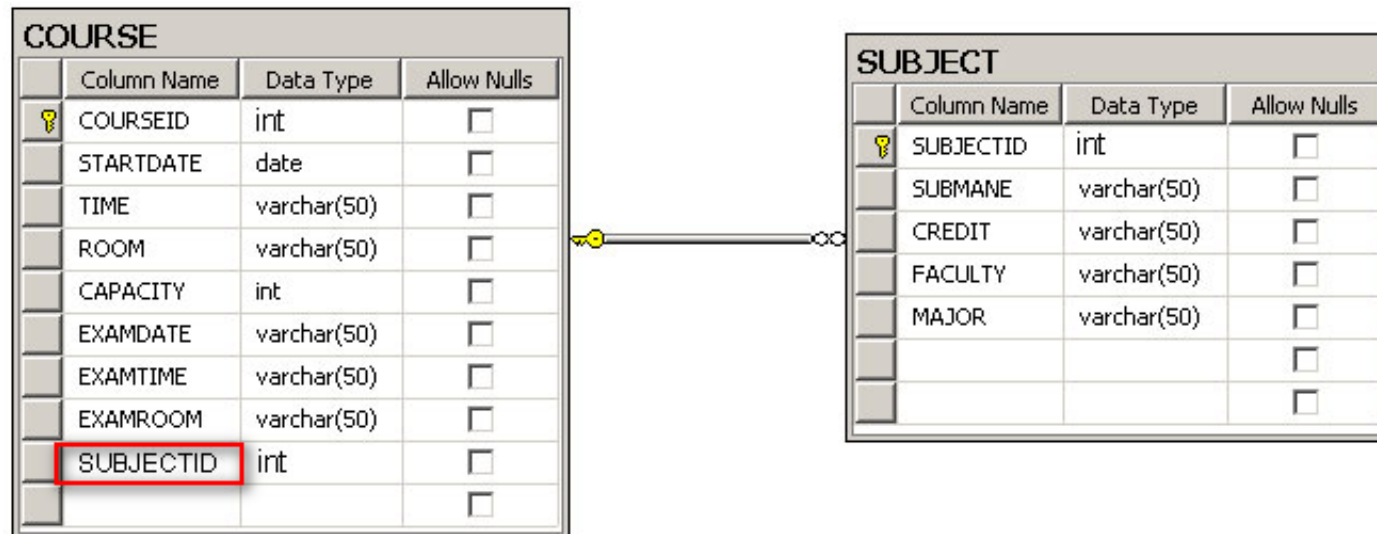
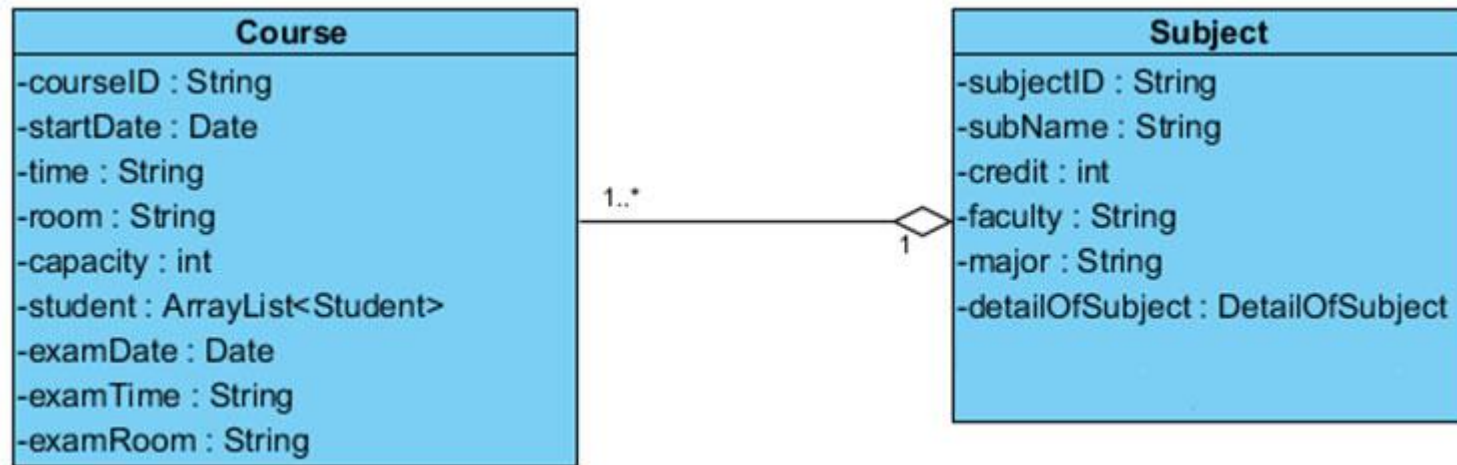
ID	NAME	PRICE	CARMODELDETAILSID
111	Grey Shadow	19500	37
39	Fly	25000	19
14	Dooby 9	35000	18

CARMODELDETAILS

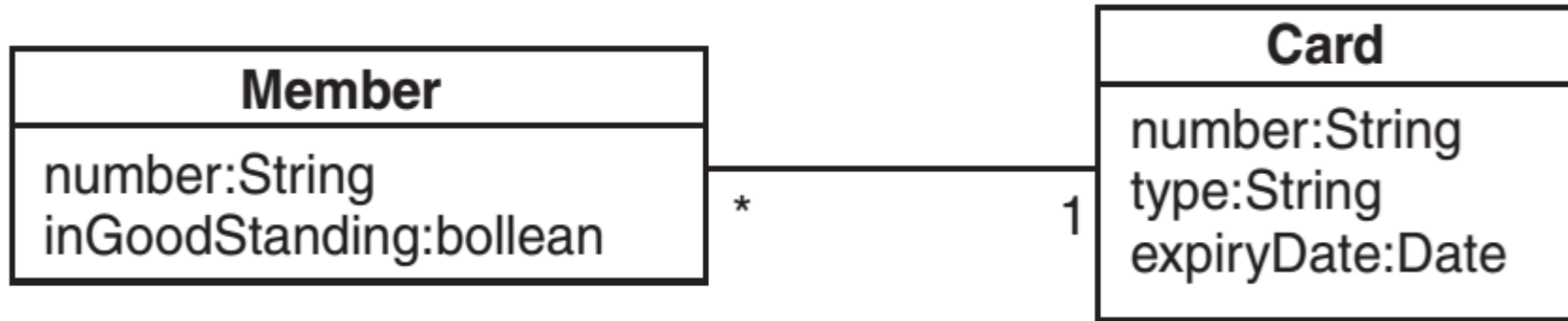
ID	ENGINESIZE	DESCRIPTION	ADVERT	POSTER
19	3500	Pure luxury...	arf.ram	arf.jpg
18	3000	Power and...	amd9.ram	amd9.jpg
37	4800	Smooth but...	rcgs.ram	rcgs.jpg

| Liên kết 1-n

- Với liên kết 1-n, chúng ta có thể thêm một khóa ngoại vào bảng “nhiều” như hình dưới. Vì một **Subject** có thể có nhiều **Course** nên **SUBJECTID** được thêm vào bảng **Course**.



| Liên kết 1-n (Ví dụ)



CARD

ID	TYPE	NUMBER	EXPIRYDATE
14	Visor	1111 2222 3333 4444	2006-10-09
45	Annex	7777 8888 9999 5555	2006-12-14

MEMBER

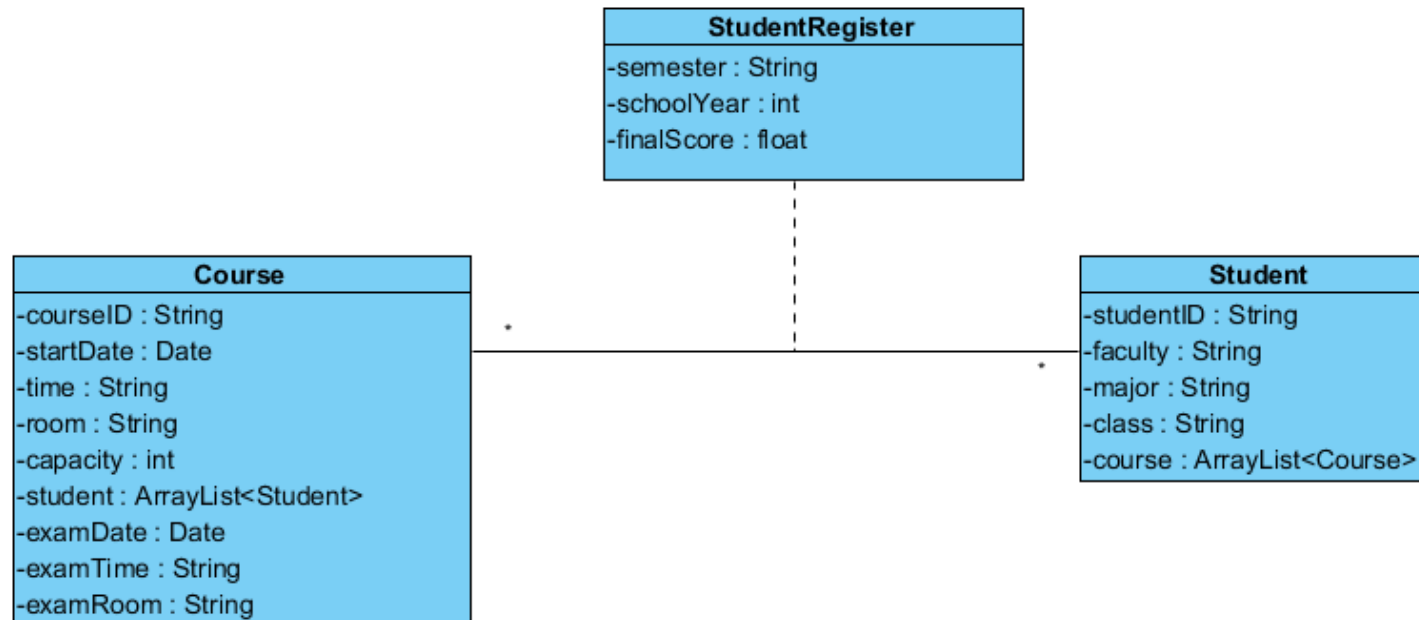
ID	NUMBER	CARDID	INGOODSTANDING
4	M105	14	TRUE
11	M9371	45	FALSE
2	M203	14	TRUE

| Liên kết n-m

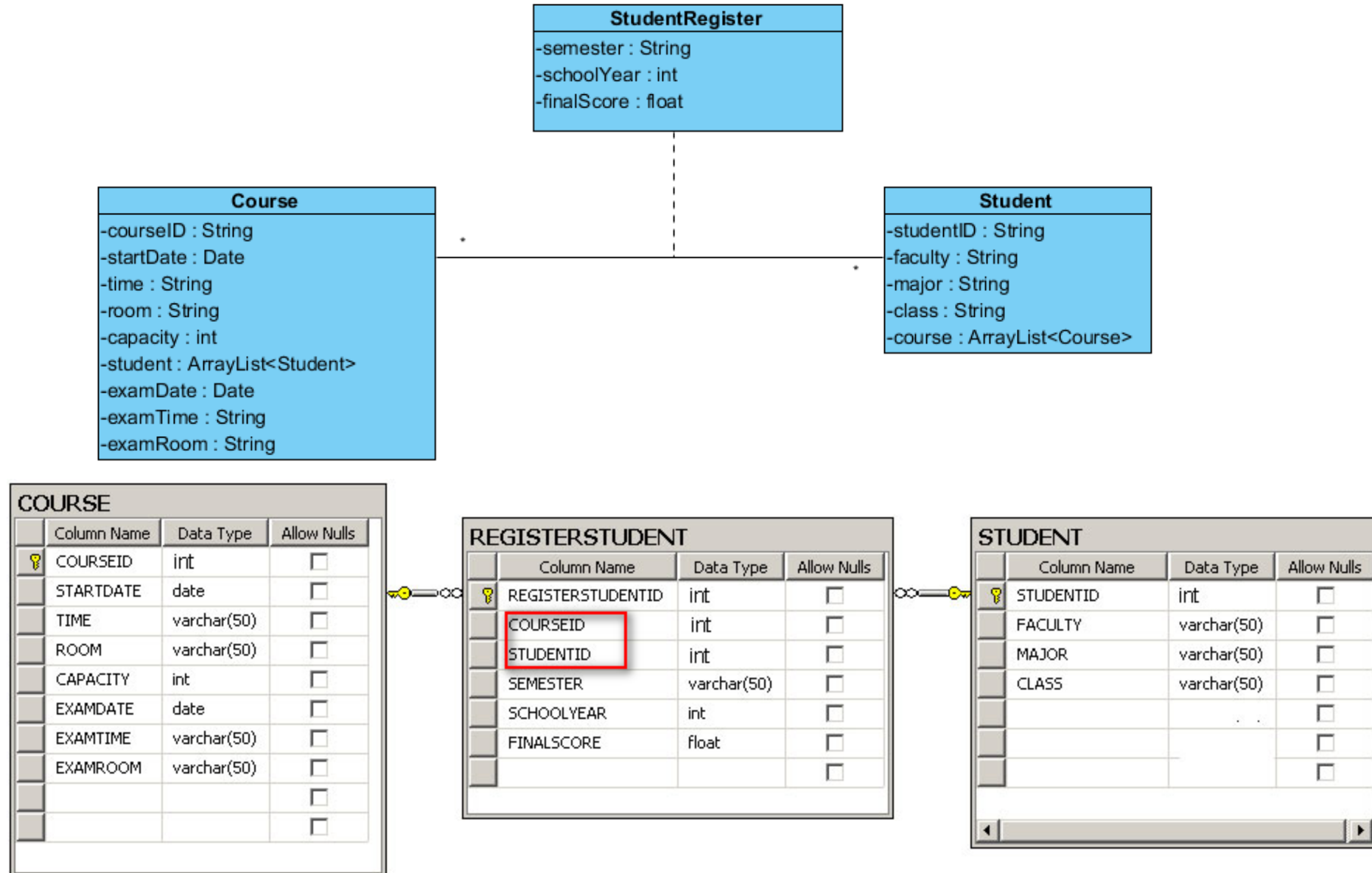
- Đối với quan hệ **nhiều-nhiều**, một khóa ngoại là chưa đủ để định danh nhiều thực thể ở mỗi đầu liên kết. Cách giải quyết là sử dụng một bảng liên kết, trong đó mỗi dòng trong bảng liên kết biểu diễn một kết nối giữa một thực thể trong một bảng và một thực thể trong bảng khác.
- Bảng liên kết có khoá chính là hợp của hai khoá ngoại ánh xạ đến bảng. Chúng ta có thể sử dụng bảng liên kết để lưu trữ quan hệ **một - một**, **một - nhiều**. Quan hệ **n-m** có thể xem là trường hợp đặc biệt của các lớp liên kết.

| Các lớp liên kết

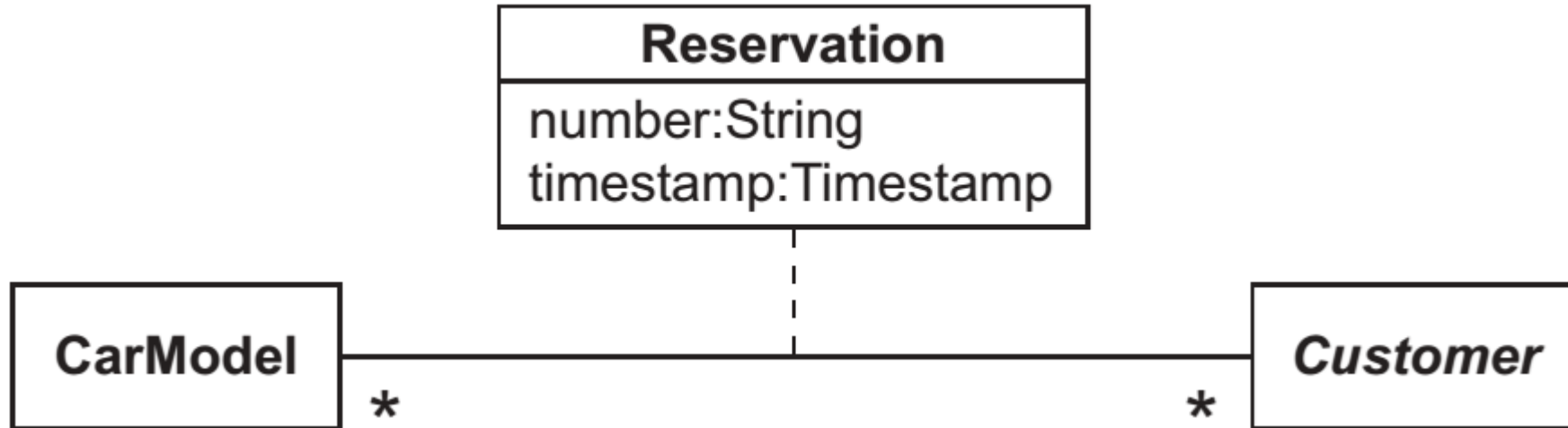
- Các lớp liên kết có dữ liệu riêng nên cần được ánh xạ thành bảng liên kết.
- Bảng biểu diễn lớp liên kết có cột chứa thuộc tính của nó và có thể có một cột ID làm khoá chính nếu lớp liên kết biểu diễn một thực thể.
- Hình sau biểu diễn bảng cho lớp liên kết **StudentRegister**. Bảng **STUDENTREGISTER** có khóa chính cho nó và hai khóa ngoại **STUDENTID** và **COURSEID**.



| Các lớp liên kết



| Lớp liên kết (Ví dụ)

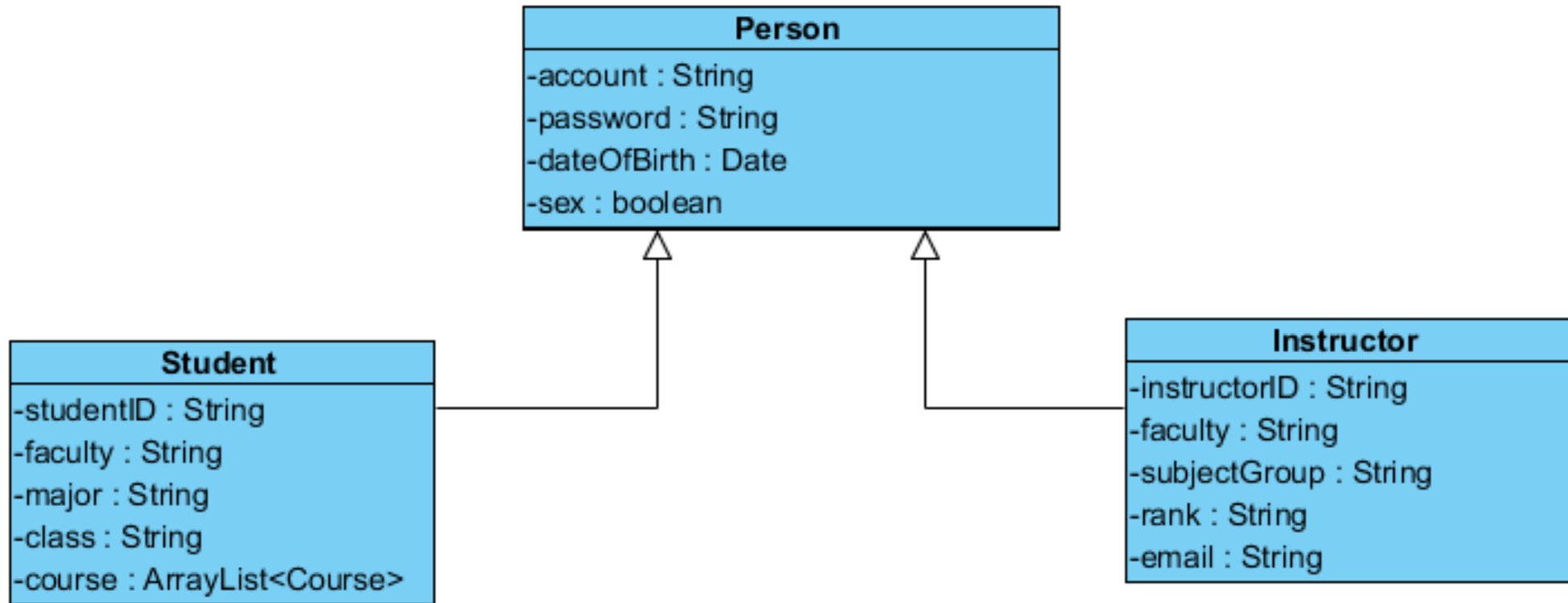


RESERVATION

ID	CARMODELID	CUSTOMERID	NUMBER	TIMESTAMP
7	33	4	R187a	2004-12-06 14:23:16.543
1	22	2	R7b	2004-12-03 00:03:21.872
99	72	4	R459b	2004-12-05 09:45:07.210

| 6.3.4 Ánh xạ kế thừa

- Xét ví dụ sau:



| Có ba cách để ánh xạ quan hệ kế thừa sang CSDL quan hệ.

Cách 1: Sử dụng một bảng cho lớp thực thể cha và tất cả các thuộc tính của lớp con được lưu trữ trong một bảng.

- Thêm trường **Type** để phân biệt. Ví dụ Type=1 là Student, Type=2 là Instructor ...
- Hoặc thêm trường **Discriminator**. Discriminator có thể là “Student” hoặc “Instructor”.
- Cách này có ưu điểm là đơn giản và tính đa hình được hỗ trợ.
- Tuy nhiên, nó có nhược điểm là mỗi lần thêm một thuộc tính mới vào bất kì bảng nào trong quan hệ kế thừa đều phải thêm một cột vào bảng và do đó dễ tăng sự trùng lặp trong phân cấp lớp.

PERSON			
	Column Name	Data Type	Allow Nulls
🔑	ID	int	☐
	ACCOUNT	varchar(50)	☐
	PASSWORD	varchar(50)	☐
	DATEOFBIRTH	date	☐
	SEX	bit	☐
	FACULTY	varchar(50)	☐
	MAJOR	varchar(50)	☐
	CLASS	varchar(50)	☐
	COURSEID	int	☐
	SUBJECTGROUP	varchar(50)	☐
	RANK	varchar(50)	☐
	EMAIL	varchar(50)	☐
	FULLNAMEID	int	☐
	ADDRESSID	int	☐
	TYPE	int	☐

| Có ba cách để ánh xạ quan hệ kế thừa sang CSDL quan hệ.

Cách 2: Sử dụng một bảng cho một lớp con cụ thể, mỗi bảng chứa cả thuộc tính của lớp cha và của chính chúng.

- Cách này có lợi điểm là dữ liệu của đối tượng chỉ lưu trữ trong một bảng, không phải di chuyển qua các bảng để lấy dữ liệu.
- Tuy nhiên, có nhược điểm là khi thêm một thuộc tính vào lớp cha thì phải sửa ở tất cả các bảng là lớp con của nó.

STUDENT			
	Column Name	Data Type	Allow Nulls
🔑	STUDENTID	int	☐
	FACULTY	varchar(50)	☐
	MAJOR	varchar(50)	☐
	CLASS	varchar(50)	☐
	COURSEID	varchar(50)	☐
	RESULTSTUDENTID	varchar(50)	☐
	ACCOUNT	varchar(50)	☐
	PASSWORD	varchar(50)	☐
	DATEOFBIRTH	date	☐
	SEX	bit	☐
	FULLNAMEID	int	☐
	ADDRESSID	int	☐
			☐

INSTRUCTOR			
	Column Name	Data Type	Allow Nulls
🔑	INSTRUCTORID	int	☐
	FACULTY	varchar(50)	☐
	SUBJECTGROUP	varchar(50)	☐
	RANK	varchar(50)	☐
	EMAIL	varchar(50)	☐
	ACCOUNT	varchar(50)	☐
	PASSWORD	varchar(50)	☐
	DATEOFBIRTH	date	☐
	SEX	bit	☐
	FULLNAMEID	int	☐
	ADDRESSID	int	☐
			☐

| Có ba cách để ánh xạ quan hệ kế thừa sang CSDL quan hệ.

Cách 3: Sử dụng mỗi bảng cho mỗi lớp. Khóa chính của lớp cha cũng sẽ là khóa chính cho cả hai lớp con. Nghĩa là những đối tượng vừa thuộc lớp cha **Person** và lớp **Instructor** sẽ có khóa giống nhau và vừa thuộc lớp cha **Person** và **Student** sẽ có khóa giống nhau.

- Lợi điểm của cách này là thoả mãn khái niệm hướng đối tượng nhất và hỗ trợ tính đa hình, hơn nữa dễ dàng sửa lớp cha và thêm các lớp con mới.
- Tuy nhiên, nhược điểm là có quá nhiều bảng và mất nhiều thời gian để đọc và ghi dữ liệu do phải xử lý trên nhiều bảng.

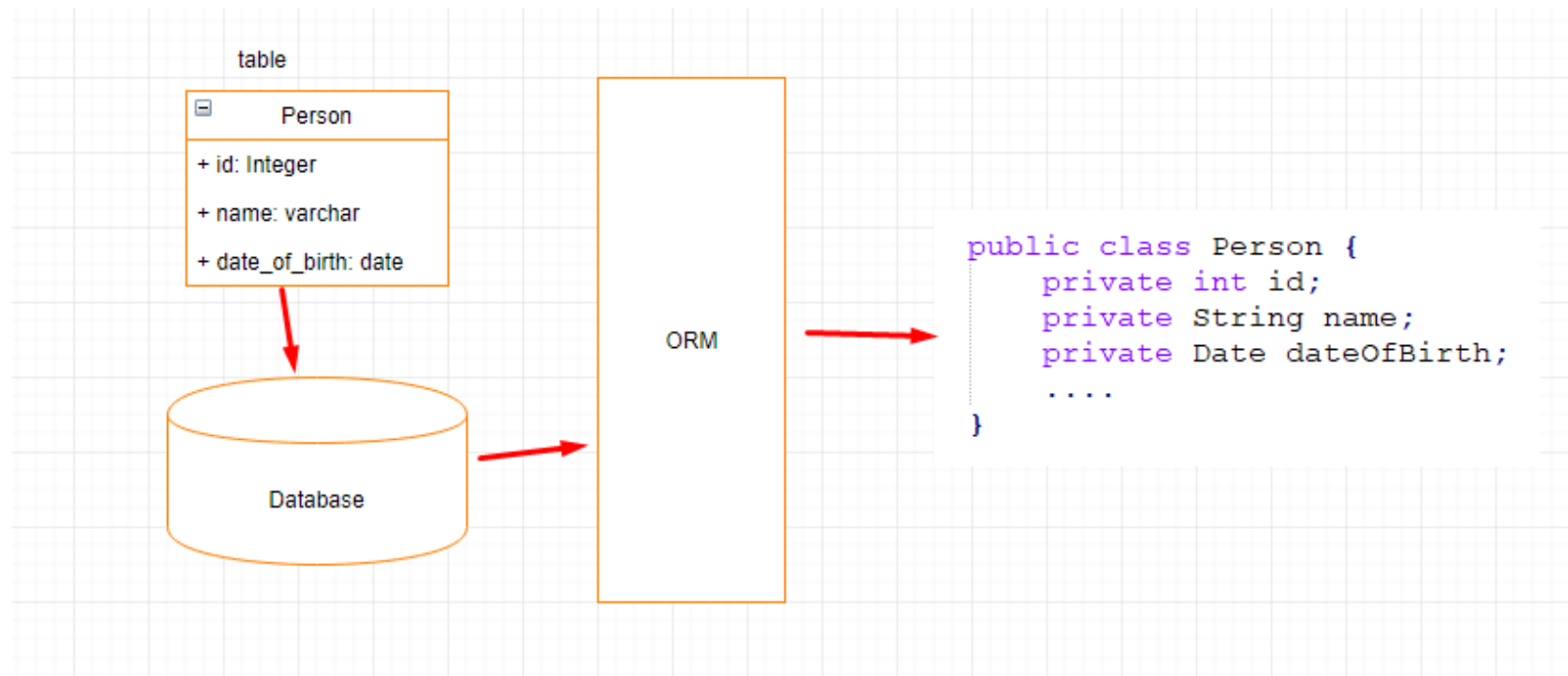
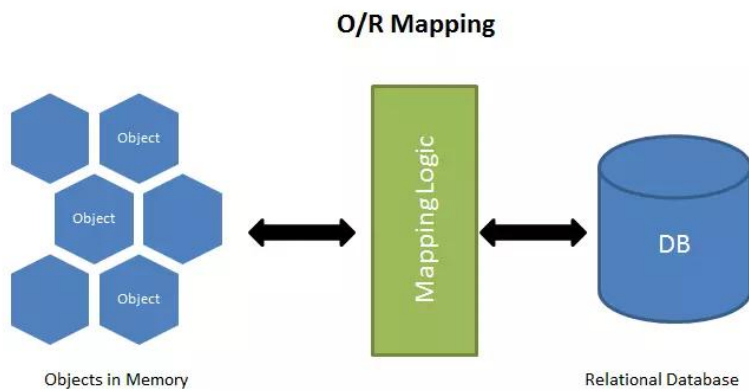
PERSON			
	Column Name	Data Type	Allow Nulls
🔑	ID	int	☐
	ACCOUNT	varchar(50)	☐
	PASSWORD	varchar(50)	☐
	DATEOFBIRTH	date	☐
	SEX	bit	☐
	FULLNAMEID	varchar(50)	☐
	ADDRESSID	int	☐

INSTRUCTOR			
	Column Name	Data Type	Allow Nulls
🔑	INSTRUCTORID	int	☐
	FACULTY	varchar(50)	☐
	SUBJECTGROUP	varchar(50)	☐
	RANK	varchar(50)	☐
	EMAIL	varchar(50)	☐

STUDENT			
	Column Name	Data Type	Allow Nulls
🔑	STUDENTID	int	☐
	FACULTY	varchar(50)	☐
	MAJOR	varchar(50)	☐
	CLASS	varchar(50)	☐
	COURSEID	int	☐
	RESULTSTUDENTID	int	☐

| Object Relational Mapping (ORM)

- **ORM** (Object-Relational Mapping) là một kỹ thuật lập trình tiên tiến giúp **ánh xạ cấu trúc của cơ sở dữ liệu** quan hệ phức tạp vào **các đối tượng trong phần mềm** được viết bằng các ngôn ngữ hướng đối tượng.
- **ORM** cung cấp một lớp trừu tượng (abstraction layer) giúp giảm thiểu sự phụ thuộc vào các truy vấn SQL phức tạp, đơn giản hóa việc thao tác với dữ liệu và nâng cao tính bảo trì của code.



| Object Relational Mapping (ORM)

- **Ưu điểm:**

- **Tăng hiệu suất:** giúp giảm thời gian và công sức cần thiết để viết các truy vấn dữ liệu bằng cách cung cấp một API đơn giản để tạo, đọc, cập nhật và xóa dữ liệu.
- **Cải thiện khả năng bảo trì:** Logic truy vấn dữ liệu được tách biệt khỏi code nghiệp vụ, giúp code dễ đọc, dễ hiểu và dễ bảo trì hơn.
- **Giảm thiểu lỗi:** ORM có thể giúp giảm lỗi do đánh máy trong các câu lệnh SQL.
- **Tính độc lập với cơ sở dữ liệu:** ORM có thể giúp code ứng dụng trở nên độc lập hơn với cơ sở dữ liệu, do ORM có khả năng ánh xạ với các hệ quản trị cơ sở dữ liệu khác nhau.

- **Nhược điểm:**

- **Mất hiệu suất:** Trong một số trường hợp, ORM có thể tạo ra các truy vấn SQL kém hiệu quả so với việc viết tay.
- **Tính linh hoạt hạn chế:** ORM có thể không hỗ trợ đầy đủ tất cả các tính năng của SQL.
- **Độ phức tạp:** ORM có thể làm tăng độ phức tạp của code đối với các nhà phát triển không quen thuộc với chúng.

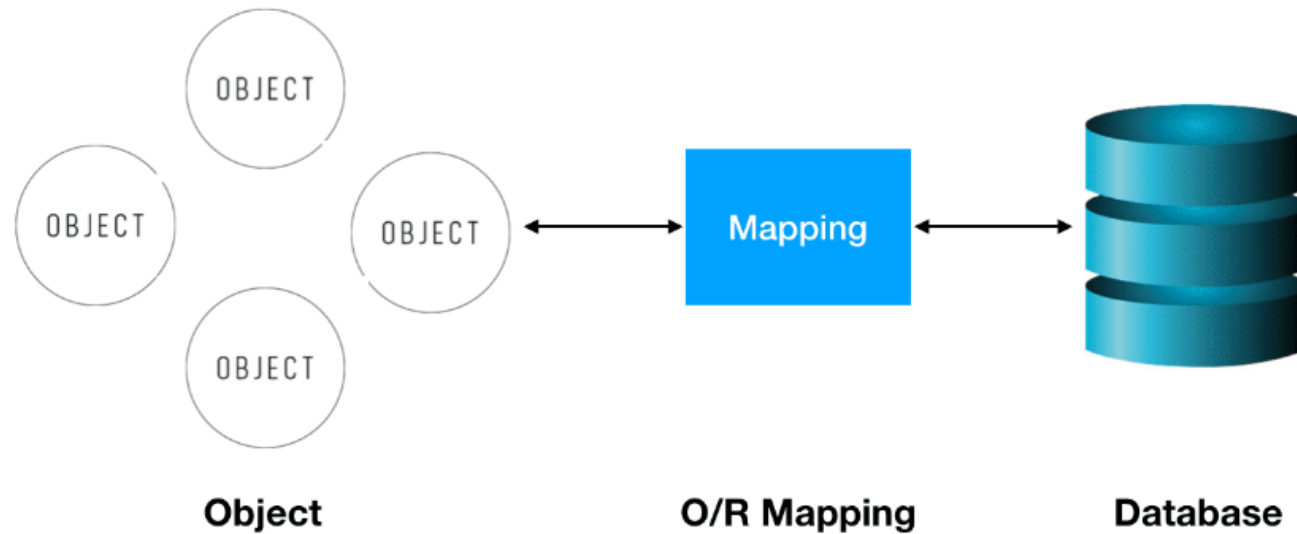
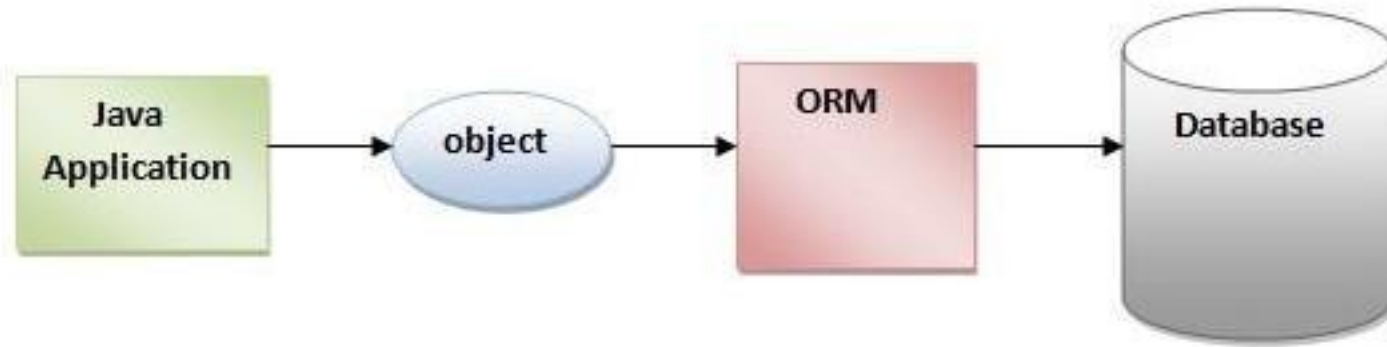
| Object Relational Mapping (ORM)

Ngôn ngữ lập trình	Thư viện
C#	NHibernate, Entity Framework
Java	JPA, Hibernate, EclipseLink, Oracle TopLink, OpenJPA
PHP	Propel, Doctrine, Eloquent, RedBean
Python	SQLAlchemy, Django
Ruby	ActiveRecord, DataMapper
Node.js	Bookshelf, Knex, Objection, Sequelize, Waterline
Golang	GORM

| Java Persistence API

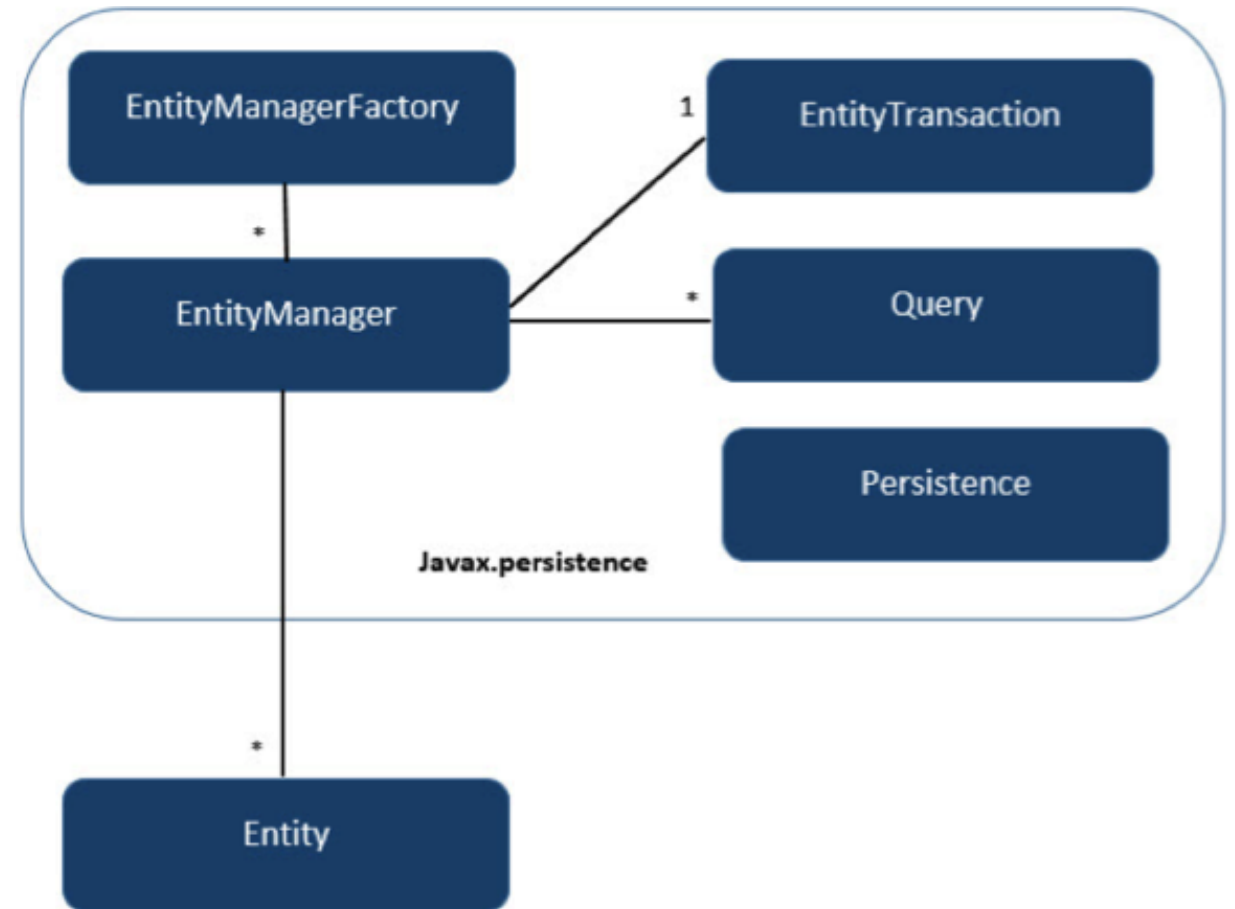
- JPA là một đặc tả Java cho việc ánh xạ giữa các đối tượng Java với cơ sở dữ liệu quan hệ sử dụng công nghệ phổ biến là ORM (Object Relational Mapping).
 - JPA cung cấp đầy đủ các công cụ cho phép chúng ta có thể thao tác với cơ sở dữ liệu một cách đơn giản và nhanh chóng.
 - JPA hoạt động như một cầu nối giữa các table/các mối quan hệ giữa các table trong database và các class/ mối quan hệ giữa các object. Ví dụ: table USER với các column (Id, username, password) sẽ tương ứng với class User.java với các field Id, username, password.
- > Từ đó mỗi khi truy vấn table hay các column ta sẽ gọi trực tiếp các phương thức trên các class, các field của class mà không cần quan tâm tới việc đang dùng loại database nào, kiểu dữ liệu database ra sao, ...

| Java Persistence API



| Kiến trúc và các thành phần của JPA

- **EntityManager**: interface cung cấp các API cho việc tương tác với các Entity.
- **EntityManagerFactory**: được dùng để tạo ra một instance của EntityManager.
- **Persistence**: định nghĩa một tập hợp các Entity class được quản lý bởi 1 instance của EntityManager trong ứng dụng.
- **EntityTransaction**: có quan hệ 1-1 với EntityManager. Bất kỳ thao tác nào được bắt đầu thông qua đối tượng **EntityManager** đều được đặt trong một Transaction.
- **Query**: là một interface, được nhà cung cấp JPA implement để có được các đối tượng quan hệ đáp ứng các tiêu chí truy vấn.



| JPA Example

```
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;

@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // Getters and setters
}
```

| JPA Example

```
import javax.persistence.EntityManager;  
import javax.persistence.EntityManagerFactory;  
import javax.persistence.Persistence;  
  
public class UserService {  
    private EntityManagerFactory emf = Persistence.createEntityManagerFactory("my-persistence-unit");  
  
    public void createUser(String name, String email) {  
        EntityManager em = emf.createEntityManager();  
        em.getTransaction().begin();  
  
        User user = new User();  
        user.setName(name);  
        user.setEmail(email);  
  
        em.persist(user);  
        em.getTransaction().commit();  
        em.close();  
    }  
}
```

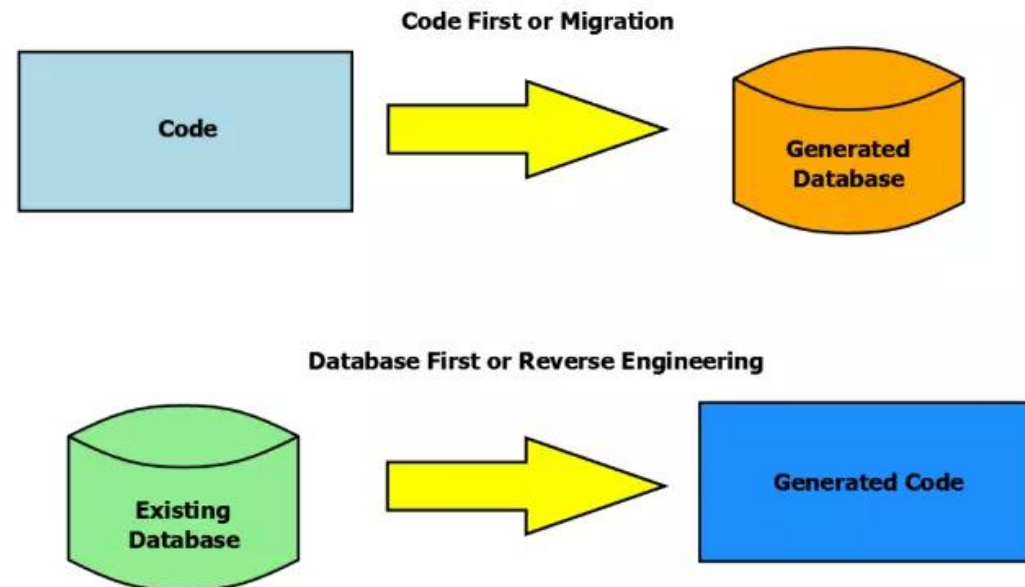
Gợi ý: Tìm hiểu thêm về **Hibernate**, **Spring Data JPA** và so sánh chúng với nhau.

| Entity Framework và Entity Framework Core

- **Entity Framework (EF)** là một công nghệ Object-Relational Mapping (ORM) được phát triển bởi Microsoft, đóng vai trò như cầu nối giữa lập trình hướng đối tượng (OOP) và cơ sở dữ liệu quan hệ. EF giúp đơn giản hóa việc truy cập và thao tác dữ liệu, cho phép nhà phát triển tập trung vào logic ứng dụng thay vì loay hoay với các truy vấn SQL phức tạp.
- **Entity Framework Core (EF Core)** là phiên bản mới của Entity Framework sau EF 6.x. Nó là mã nguồn mở, nhẹ, có thể mở rộng và là phiên bản đa nền tảng của công nghệ truy cập dữ liệu Entity Framework.
- EF Core được sử dụng với các ứng dụng .NET Core. Tuy nhiên, nó cũng có thể được sử dụng với các ứng dụng dựa trên .NET Framework 4.5+ tiêu chuẩn.

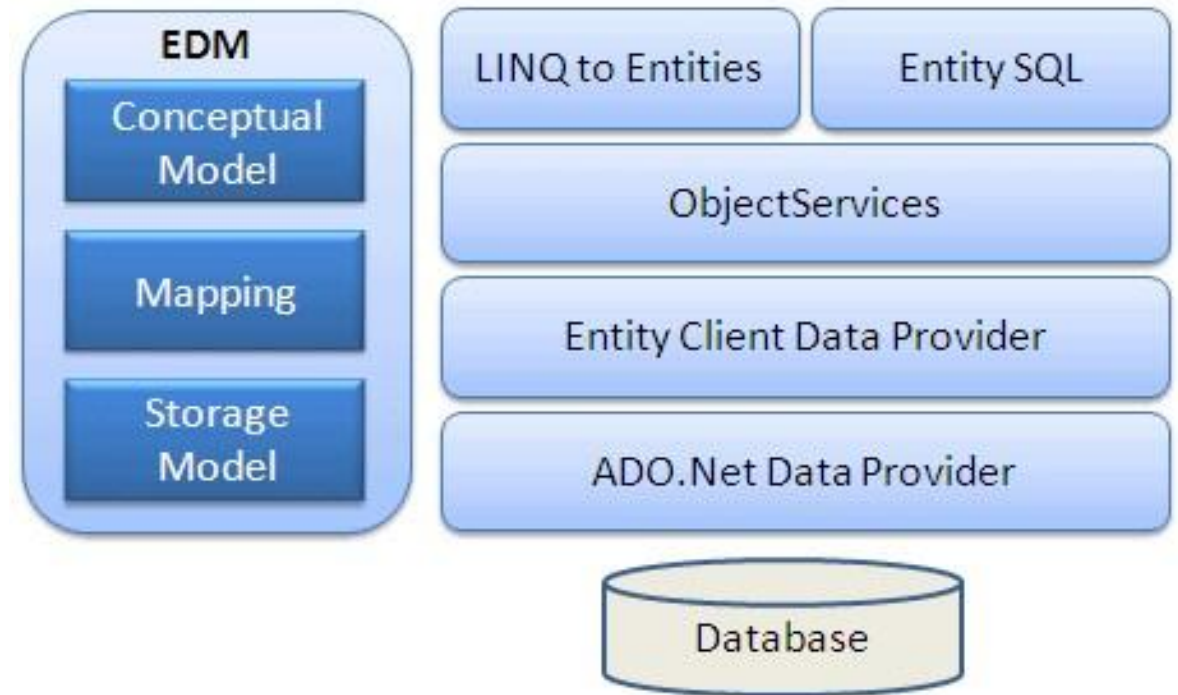
| Các hướng tiếp cận trong Entity Framework

- **Database First:** Khi bạn đã có một cơ sở dữ liệu hoặc muốn thiết kế CSDL trước khi bắt đầu phát triển những phần khác của ứng dụng. Trong tình huống này, EF cho phép bạn tạo các đối tượng lớp dựa trên cơ sở dữ liệu tồn tại.
- **Code First:** Khi bạn muốn tập trung vào việc xây dựng các class trước và sau đó sử dụng chúng để tạo ra cơ sở dữ liệu. Điều này giúp tạo ra một mô hình dữ liệu linh hoạt và tự nhiên dựa trên các đối tượng trong mã nguồn. Bên cạnh đó, EF Code First cũng giúp tập trung vào việc thiết kế mô hình dữ liệu hướng đối tượng thay vì viết các truy vấn SQL phức tạp.
- **Model First:** Bạn muốn thiết kế schema của CSDL trên visual designer rồi mới tạo CSDL và các class.



| Các thành phần của Entity Framework

- **EDM (Entity Data Model):**
 - Conceptual Model: Chứa các model class và các quan hệ của nó.
 - Mapping: gồm có thông tin về cách Conceptual model kết nối với Storage model.
 - Storage Model: Đây là database design model bao gồm các bảng, views, stored procedures, và những quan hệ và các khóa.
- **LINQ to Entities, Entity SQL :** Là ngôn ngữ truy vấn sử dụng để viết các truy vấn tới object model. Trả về các thực thể được định nghĩa bên trong Conceptual model.
- **Object Service:** sẽ có trách nhiệm trong việc cụ thể hóa quá trình chuyển đổi dữ liệu trả về từ một Entity Client Data Provider tới một Entity Object Structure.
- **Entity Client Data Provider:** chuyển đổi truy vấn Entity SQL thành một truy vấn SQL.
- **ADO.Net Data Provider:** Giao tiếp CSDL.



| Entity Framework Example

Department Table

```
CREATE TABLE [dbo].[Departments](
    [DeptUniqueId] [int] IDENTITY(1,1) NOT NULL,
    [Capacity] [int] NOT NULL,
    [DeptName] [nvarchar](20) NOT NULL,
    [DeptNo] [int] NOT NULL,
    [Location] [nvarchar](20) NOT NULL
)
```

Employee Table

```
CREATE TABLE [dbo].[Employees](
    [EmpUniqueId] [int] IDENTITY(1,1) NOT NULL,
    [DeptUniqueId] [int] NOT NULL,
    [Designation] [nvarchar](20) NOT NULL,
    [EmpFirstName] [nvarchar](80) NOT NULL,
    [EmpLastName] [nvarchar](80) NULL,
    [EmpNo] [int] NOT NULL,
    [Salary] [int] NOT NULL,
    [DeptUniqueId] int references master(Departments)
)
```



```
namespace EFCore_Scaffolding.Models
{
    8 references
    public partial class Departments
    {
        2 references
        public Departments()
        {
            Employees = new HashSet<Employees>();
        }
        3 references
        public int DeptUniqueId { get; set; }
        3 references
        public int Capacity { get; set; }
        4 references
        public string DeptName { get; set; }
        3 references
        public int DeptNo { get; set; }
        4 references
        public string Location { get; set; }
        2 references
        public virtual ICollection<Employees> Employees { get; set; }
    }
}
```

```
namespace EFCore_Scaffolding.Models
{
    4 references
    public partial class Employees
    {
        1 reference
        public int EmpUniqueId { get; set; }
        2 references
        public int DeptUniqueId { get; set; }
        1 reference
        public string Designation { get; set; }
        1 reference
        public string EmpFirstName { get; set; }
        1 reference
        public string EmpLastName { get; set; }
        0 references
        public int EmpNo { get; set; }
        0 references
        public int Salary { get; set; }
        1 reference
        public virtual Departments DeptUnique { get; set; }
    }
}
```

I BÀI TẬP NHÓM CHƯƠNG 6 – TÍNH ĐIỂM BÀI TẬP

Quy định:

- Nhóm từ 1 - 3 bạn.
- Nội dung: Hãy trình bày thiết kế chi tiết của hệ thống.
- Hạn nộp bài: 30/11/2024 (In, đóng cuốn)

Nhóm chọn 1 trong các Bài tập sau hoặc Có thể chọn 1 Hệ thống khác:

- Hệ thống Quản lý đặt chỗ thuê xe hơi.
- Hệ thống Quản lý thư viện tài liệu điện tử.
- Hệ thống Quản lý thư viện thường.
- Hệ thống Quản lý cửa hàng bán sách.
- Hệ thống Quản lý siêu thị điện máy.

-> Bài tập này tự làm, Lớp chọn PA lấy điểm đồ án làm điểm 1 trong các cột điểm bài tập.

